TP 1 : Pipeline réactif complet

Objectifs:

- Construire un pipeline réactif end-to-end
- Combiner Mono et Flux avec opérateurs avancés
- Gérer les erreurs et timeouts
- Tester avec StepVerifier

Contexte métier

Vous développez un système de **traitement de commandes réactif** pour une boutique en ligne. Le système doit :

1. Recevoir une commande avec une liste de Product IDs
2. Valider les IDs (format et existence)
3. Récupérer les détails des produits depuis une base de données (simulée)
4. Vérifier le stock pour chaque produit
5. Appliquer les réductions applicables
6. Calculer le prix total
7. Créer la commande finale
8. Gérer les erreurs et les timeouts

Étapes de réalisation

Étape 1 : Modélisation

Ajouter les dépendances Maven :

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-test</artifactId>
   <scope>test</scope>
</dependency>
<dependency>
   <groupId>io.projectreactor</groupId>
   <artifactId>reactor-test</artifactId>
   <scope>test</scope>
</dependency>
```

Créez les classes métier :

Product.java

```
Propriétés:
- id: String
- name: String
- price: BigDecimal (> 0)
- stock: Integer (>= 0)
- category: String
```

Order.java

```
Propriétés:
- orderId: String (généré UUID)
- productIds: List<String> (d'entrée)
- products: List<ProductWithPrice> (après traitement)
- totalPrice: BigDecimal
- discountApplied: Boolean
- createdAt: LocalDateTime
- status: OrderStatus (CREATED, VALIDATED, PROCESSING, COMPLETED, FAILED)
```

ProductWithPrice.java

```
Propriétés:
- product: Product
- originalPrice: BigDecimal
- discountPercentage: Integer (%)
- finalPrice: BigDecimal
```

OrderRequest.java

```
Propriétés:
- productIds: List<String>
- customerId: String
```

Étape 2 : Repository simulé

Créez `ProductRepository.java` (classe utilitaire, pas Spring Data) :

Fonctionnalités:

- Méthode `Mono<Product> findById(String id)` : retourne Mono.empty() si n'existe pas
- Méthode `Flux<Product> findByIds(List<String> ids)` : utilise flatMapIterable
- Méthode `Mono<Integer> getStock(String productId)` : simule une appel BDD réactif

- Contenir 5 produits en dur (en mémoire)
- Ajouter un délai de 100ms par appel pour simuler la latence DB

Bonus: Ajouter des erreurs aléatoires (10%) pour tester la gestion d'erreurs

Étape 3 : Service métier

Créez `OrderService.java` :

Méthode `processOrder(OrderRequest request): Mono<Order>`

Pipeline à implémenter :

```
1. Valider la requête (pas vide, Customer ID non-null)
   → Si invalide: Mono.error(InvalidOrderException)

2. Convertir List<String> en Flux<String>
   → filter(): éliminer les IDs vides/null
   → take(100): limiter à 100 produits max

3. Pour chaque ID:
   → flatMap vers ProductRepository.findById()
   → Si produit n'existe pas: onErrorResume() → logger + ignorer
   → filter(): vérifier que stock > 0

4. Appliquer les réductions:
   → map(): pour chaque produit, calculer la remise
     * Électronique (catégorie): 10%
     * Autres: 5%
   → Créer ProductWithPrice avec prix final

5. Combiner les résultats:
   → collectList(): rassembler tous les ProductWithPrice
   → map(): créer l'Order finale avec totalPrice, status=COMPLETED

6. Gestion d'erreurs:
   → timeout(5 seconds): lever une exception après 5s
   → onErrorResume(Exception e): créer Order avec status=FAILED
   → doOnError(): logger l'erreur

7. Logging détaillé:
   → doOnNext(): log chaque étape
   → doOnError(): log chaque erreur
   → doFinally(): log la fin du processus
```

Étape 4 : Tests

Créez `OrderServiceTest.java` dans `src/test/java/` :

Tests à implémenter:

1. **test_processOrderSuccess()** - Cas nominal
    - Créer OrderRequest avec IDs valides existants
    - Vérifier que l'Order est créée
    - Vérifier que totalPrice est correctement calculé
    - Vérifier que status = COMPLETED
    - Utiliser StepVerifier
2. **test_processOrderWithInvalidIds()** - IDs invalides mélangés
    - Mélanger IDs valides et invalides
    - Vérifier que seuls les valides sont traités
    - Vérifier que Order contient N produits (N < taille liste origine)
3. **test_processOrderWithoutStock()** - Produits out of stock
    - Utiliser produits avec stock = 0
    - Vérifier qu'ils ne sont pas inclus dans l'Order
4. **test_processOrderWithDiscounts()** - Vérifier les réductions
    - Mélanger catégories (électro et autres)
    - Vérifier que réduction électro = 10%
    - Vérifier que réduction autres = 5%
    - Vérifier que totalPrice = sum(finalPrice)
5. **test_processOrderTimeout()** - Timeout après 5s
    - Créer ProductRepository avec délai > 5s
    - Vérifier que timeout est levé
    - Vérifier que Order.status = FAILED
6. **test_processOrderWithErrors()** - Gestion des erreurs
    - Simuler 50% d'erreurs du repository
    - Vérifier que onErrorResume fonctionne
    - Vérifier que Order est créée même avec erreurs partielles

Exemple de test avec StepVerifier:

```
@Test
void test_processOrderSuccess() {
    OrderRequest request = new OrderRequest(
        Arrays.asList("PROD001", "PROD002"),
        "CUST001"
    );

    StepVerifier.create(orderService.processOrder(request))
        .assertNext(order -> {
            assertThat(order.getOrderId()).isNotNull();
            assertThat(order.getProducts()).hasSize(2);
```

```
            assertThat(order.getTotalPrice()).isGreaterThan(BigDecimal.ZERO);
            assertThat(order.getStatus()).isEqualTo(OrderStatus.COMPLETED);
        })
        .verifyComplete();
}
```

Étape 5 : Optimisations bonus

Implémentez au moins UNE des optimisations :

Option A: Parallélisation

- Utiliser `parallel()` et `Schedulers.parallel()` pour traiter les produits en parallèle
- Mesurer l'amélioration de temps d'exécution

Option B: Caching

- Ajouter un cache simple (Map<String, Mono>) pour les produits récemment consultés
- Éviter les requêtes redondantes

Option C: Circuit Breaker

- Intégrer Resilience4j pour ajouter un Circuit Breaker autour du ProductRepository
- Éviter les appels répétés en cas de défaillance

Option D: Batching

- Grouper les requêtes de produits par batch (ex: 10 à la fois)
- Réduire le nombre de requêtes DB

Livrables attendus

- ☑ Fichier `ProductRepository.java` avec 5 méthodes
- ☑ Fichier `OrderService.java` avec pipeline réactif complet
- ☑ Fichier `OrderServiceTest.java` avec 6 tests minimum
- ☑ Classes métier complètes (Product, Order, ProductWithPrice, OrderRequest)
- ☑ Tous les tests passent ✓
- ☑ Au moins UNE optimisation bonus implémentée
- ☑ Documentation du pipeline (commentaires ou README.md)